Data Pipeline on AWS

Medallion Architecture · Bronze → Silver → Gold

ShopBR · E-commerce · AWS · us-east-1

Amazon S3 · AWS Lambda · AWS Glue · Amazon Athena · CloudWatch

Joyce Mara · 2026

The challenge

Contex

What was requested

Build a data pipeline on AWS for the fictional e-commerce site ShopBR, following the Medallion Architecture pattern (Bronze → Silver → Gold).

The data arrived in unprocessed CSV files. The goal was to automate the ingestion, cleaning, transformation, and availability of the data for analysis, without manual intervention.

What the business team wanted to know

- Which products sold the most last month?
- Which region of Brazil generates the most revenue?
- Which orders are currently overdue?
- What is the average order value per product category?

The solution: Medallion Architecture

Overview

Medallion Architecture organizes data into three progressive layers, each with a specific purpose.

Bronze

Ingestion

Raw data, preserved exactly as it arrived. No alterations. Ensures complete traceability of origin.

Silver

Processing

Cleaned data, type assigned, and transformed. Nulls removed, dates converted, partitioned by state for performance.

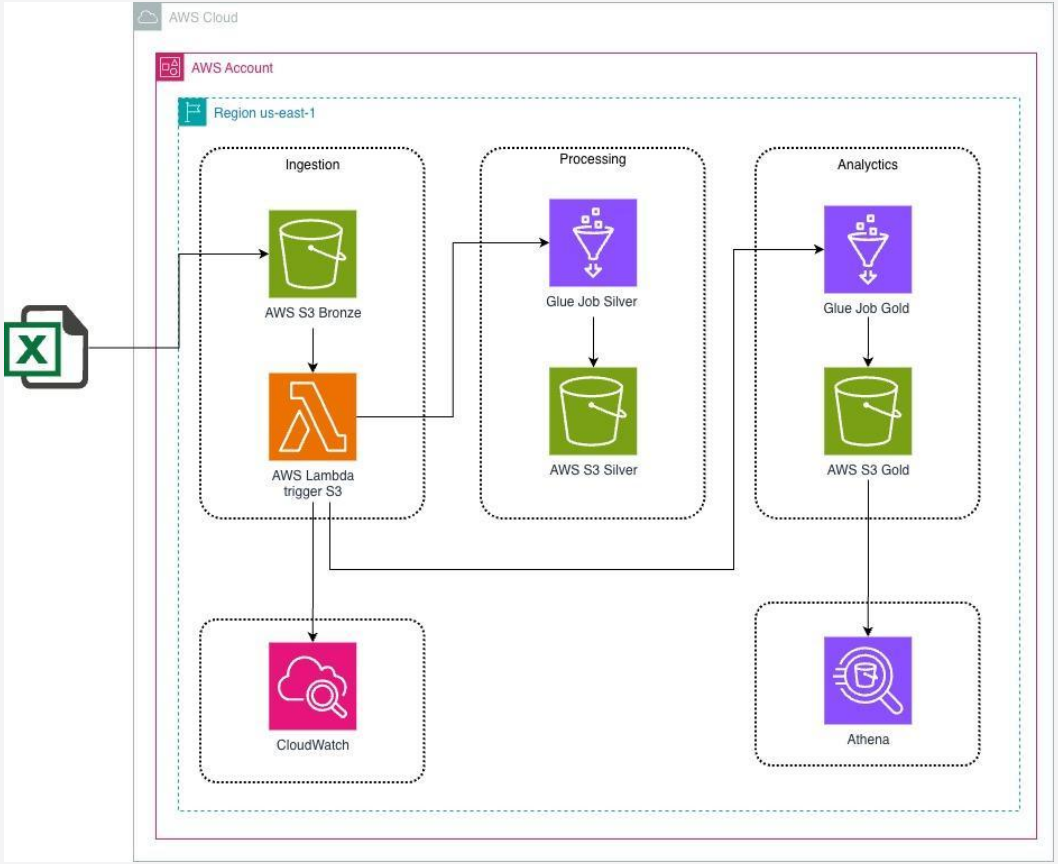
Gold

Analytics

Aggregated data, ready for analysis. Sales by state and month. Directly answers business questions.

Architecture

Overview



Bronze Layer: Ingestion

Ingestion

What happens in this layer?

The CSV file arrives in Amazon S3 in the bronze/pedidos/ folder. No transformation is performed; the data is preserved exactly as it was received. This ensures traceability: if something goes wrong during processing, it's always possible to reprocess it from the original data.

AWS S3

Stores the raw CSVs in bronze/pedidos/. Bucket: shopbr-datalake-joyce.

AWS Lambda

It monitors the bucket. When it detects a new file via the s3:ObjectCreated event, it automatically triggers the pipeline.

AWS CloudWatch

It records all execution logs. This allows you to track each stage of the pipeline and identify failures.

Amazon S3: Bucket shopbr-datalake-joyce

Ingestion

shopbr-datalake-joyce Info

Objects | Metadata | Properties | Permissions | Metrics | Management | File systems - *new* | Access Points

Objects (5)

Copy S3 URI | Copy URL | Download | Open | Delete | Actions | Create folder | Upload

Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more

Find objects by prefix

Show versions

1

Name	Type	Last modified	Size	Storage class
bronze/	Folder	-	-	-
gold_$folder$	-	April 14, 2026, 12:39:53 (UTC+01:00)	0 B	Standard
gold/	Folder	-	-	-
silver_$folder$	-	April 12, 2026, 10:39:01 (UTC+01:00)	0 B	Standard
silver/	Folder	-	-	-

Amazon S3: bronze/pedidos/

Ingestion

pedidos/

Copy S3 URI

Objects | Properties

Objects (2)

Copy S3 URI | Copy URL | Download | Open | Delete | Actions | Create folder | Upload

Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more

Find objects by prefix

Show versions

1

Name	Type	Last modified	Size	Storage class
pedidos_jan2024.csv	csv	April 6, 2026, 11:57:39 (UTC+01:00)	594.0 B	Standard
testando_lambda5.csv	csv	April 14, 2026, 12:33:56 (UTC+01:00)	1.2 KB	Standard

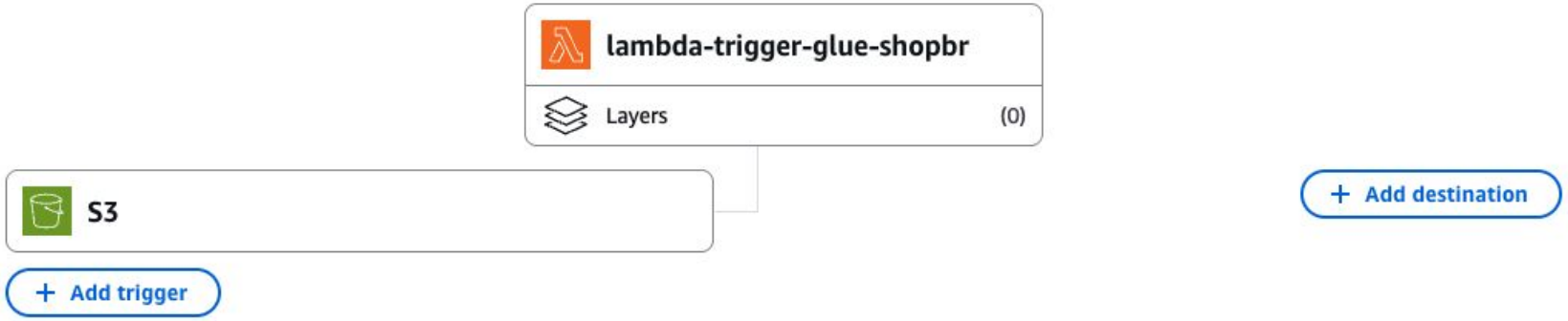
AWS Lambda: lambda-trigger-glue-shopbr

Ingestion

lambda-trigger-glue-shopbr

Function overview Info

Diagram | Template



Amazon CloudWatch: Full pipeline logs

Ingestion

CloudWatch

Favorites and recents

Ingestion
Dashboards
Alarms 0 0 0

- AI Operations
- GenAI Observability
- Application Signals (APM)
- Infrastructure Monitoring
- Logs
  - Log Management
  - Log Analytics Preview
  - Log Anomalies
  - Live Tail
  - Logs Insights
  - Contributor Insights
- Metrics
- Network Monitoring
- Setup

Log events

Actions | Start tailing | Create metric filter

You can use the filter bar below to search for and match terms, phrases, or values in your log events. Learn more about filter patterns

Filter events - press enter to search

Clear 1m 30m 1h 12h Custom UTC timezone Display

Timestamp	Message
	No older events at this moment. Retry
2026-04-14T11:33:56.765Z	INIT_START Runtime Version: python:3.14.v38 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:6198fb9c708e4801023245984069 0d2264ba8cc36621c703d5cd31b2245e02d1
2026-04-14T11:33:57.110Z	START RequestId: c69f5978-7d0a-4890-8ca5-4a6bb838f889 Version: $LATEST
2026-04-14T11:33:57.268Z	[INFO] 2026-04-14T11:33:57.268Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 Found credentials in environment variables.
2026-04-14T11:33:59.266Z	[INFO] 2026-04-14T11:33:59.266Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 Novo arquivo detectado: s3://shopbr-datalake-joyce/bronze/pedidos/testando_lambda5.csv
2026-04-14T11:33:59.671Z	[INFO] 2026-04-14T11:33:59.671Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 Job Silver iniciado: jr_0cf51b1159284bc4d11a94596098d747ace047d701a5ab53148d27e680bc5a99
2026-04-14T11:33:59.713Z	[INFO] 2026-04-14T11:33:59.713Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 [job] Estado: RUNNING | 0s decorridos
2026-04-14T11:34:29.785Z	[INFO] 2026-04-14T11:34:29.785Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 [job] Estado: RUNNING | 30s decorridos
2026-04-14T11:34:59.850Z	[INFO] 2026-04-14T11:34:59.850Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 [job] Estado: RUNNING | 60s decorridos
2026-04-14T11:35:29.924Z	[INFO] 2026-04-14T11:35:29.924Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 [job] Estado: SUCCEEDED | 90s decorridos
2026-04-14T11:35:29.924Z	[INFO] 2026-04-14T11:35:29.924Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 Job Silver concluído!
2026-04-14T11:35:39.665Z	[INFO] 2026-04-14T11:35:39.665Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 Crawler Silver iniciado!
2026-04-14T11:35:39.706Z	[INFO] 2026-04-14T11:35:39.706Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 [crawler-shopbr-silver] Estado: RUNNING | 0s decorridos
2026-04-14T11:36:09.750Z	[INFO] 2026-04-14T11:36:09.750Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 [crawler-shopbr-silver] Estado: RUNNING | 30s decorridos
2026-04-14T11:36:39.796Z	[INFO] 2026-04-14T11:36:39.796Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 [crawler-shopbr-silver] Estado: RUNNING | 60s decorridos
2026-04-14T11:37:09.838Z	[INFO] 2026-04-14T11:37:09.838Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 [crawler-shopbr-silver] Estado: STOPPING | 90s decorridos
2026-04-14T11:37:39.892Z	[INFO] 2026-04-14T11:37:39.892Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 [crawler-shopbr-silver] Estado: STOPPING | 120s decorridos
2026-04-14T11:38:09.939Z	[INFO] 2026-04-14T11:38:09.939Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 [crawler-shopbr-silver] Estado: STOPPING | 150s decorridos
2026-04-14T11:38:39.986Z	[INFO] 2026-04-14T11:38:39.986Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 [crawler-shopbr-silver] Estado: READY | 180s decorridos
2026-04-14T11:38:39.986Z	[INFO] 2026-04-14T11:38:39.986Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 Crawler Silver concluído!
2026-04-14T11:38:40.122Z	[INFO] 2026-04-14T11:38:40.122Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 Job Gold iniciado: jr_ec06c142a3a9b0ef64f4c1a54ff71cb4dc8d00d873dbf8256ca2cce00280f534
2026-04-14T11:38:40.161Z	[INFO] 2026-04-14T11:38:40.161Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 [jobGold] Estado: RUNNING | 0s decorridos
2026-04-14T11:39:10.225Z	[INFO] 2026-04-14T11:39:10.225Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 [jobGold] Estado: RUNNING | 30s decorridos
2026-04-14T11:39:40.286Z	[INFO] 2026-04-14T11:39:40.286Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 [jobGold] Estado: RUNNING | 60s decorridos
2026-04-14T11:40:10.327Z	[INFO] 2026-04-14T11:40:10.327Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 [jobGold] Estado: SUCCEEDED | 90s decorridos
2026-04-14T11:40:10.327Z	[INFO] 2026-04-14T11:40:10.327Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 Job Gold concluído!
2026-04-14T11:40:11.906Z	[INFO] 2026-04-14T11:40:11.906Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 Crawler Gold iniciado!
2026-04-14T11:40:11.952Z	[INFO] 2026-04-14T11:40:11.951Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 [crawler-shopbr-gold] Estado: RUNNING | 0s decorridos
2026-04-14T11:40:42.012Z	[INFO] 2026-04-14T11:40:42.012Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 [crawler-shopbr-gold] Estado: RUNNING | 30s decorridos
2026-04-14T11:41:12.056Z	[INFO] 2026-04-14T11:41:12.056Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 [crawler-shopbr-gold] Estado: RUNNING | 60s decorridos
2026-04-14T11:41:42.100Z	[INFO] 2026-04-14T11:41:42.100Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 [crawler-shopbr-gold] Estado: STOPPING | 90s decorridos
2026-04-14T11:42:12.142Z	[INFO] 2026-04-14T11:42:12.142Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 [crawler-shopbr-gold] Estado: STOPPING | 120s decorridos
2026-04-14T11:42:42.176Z	[INFO] 2026-04-14T11:42:42.176Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 [crawler-shopbr-gold] Estado: READY | 150s decorridos
2026-04-14T11:42:42.176Z	[INFO] 2026-04-14T11:42:42.176Z c69f5978-7d0a-4890-8ca5-4a6bb838f889 Crawler Gold concluído!
2026-04-14T11:42:42.187Z	END RequestId: c69f5978-7d0a-4890-8ca5-4a6bb838f889
2026-04-14T11:42:42.187Z	REPORT RequestId: c69f5978-7d0a-4890-8ca5-4a6bb838f889 Duration: 525076.58 ms Billed Duration: 525418 ms Memory Size: 128 MB Max Memory Used: 93 MB Init Duration: 341.34 ms
	No newer events at this moment. Auto retry paused. Resume

Back to top

Silver Layer: Processing

Processing

What happens in this layer?

The Glue Job Silver reads the CSVs from Bronze, applies transformations via PySpark, and saves the clean data in Parquet format on the Silver layer, partitioned by state. Partitioning is an important technical decision: Athena charges per scanned data point when partitioning; a SP query only reads the state=SP partition.

Remove nulls

Lines with a null order_id or value are discarded. Invalid data does not proceed to analysis.

Filter out canceled items

Orders with a canceled status are removed. Silver only reflects active operational data.

Converts types

value → double, order_date → DATE, state → uppercase. Typed and consistent schema.

Parquet + partition

Save to a state-partitioned Parquet file. Reduce CSV size from 100 MB to ~15 MB. Cost reduction of up to 90% in Athena.

AWS Glue Job

Processing

AWS Glue Studio Info

Create job Info

Author in a visual interface focused on data flow.
Visual ETL

Author using an interactive code notebook.
Notebook

Author code with a script editor.
Script editor

Example jobs Info

Create example job

Your jobs (2) Info

Actions | Run job

Filter jobs by property

1

Job name	Type	Created by	Last modified	AWS Glue version	Action
jobGold	Glue ETL	Script	12/04/2026, 12:35:59	5.0	Upgrade with AI
job	Glue ETL	Script	12/04/2026, 12:23:29	5.0	Upgrade with AI

AWS Glue Job Silver: Script PySpark

Processing

job

Script | Job details | Runs | Data quality | Schedules | Version Control

Script Info

```python
import sys
from awsglue.context import GlueContext
from pyspark.context import SparkContext
from pyspark.sql.functions import col, upper, to_date
from pyspark.sql.types import StructType, StructField, StringType, DoubleType

sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session

schema = StructType([
    StructField("pedido_id", StringType(), True),
    StructField("cliente_id", StringType(), True),
    StructField("produto", StringType(), True),
    StructField("categoria", StringType(), True),
    StructField("valor", DoubleType(), True),
    StructField("status", StringType(), True),
    StructField("estado", StringType(), True),
    StructField("data_pedido", StringType(), True)
])

df_raw = spark.read.option("header", "true").schema(schema).csv("s3://shopbr-datalake-joyce/bronze/pedidos/")

df_clean = df_raw \
    .dropna(subset=["pedido_id", "valor"]) \
    .filter(col("status") != "cancelado") \
    .withColumn("valor", col("valor").cast("double")) \
    .withColumn("estado", upper(col("estado"))) \
    .withColumn("data_pedido", to_date(col("data_pedido"), "yyyy-MM-dd"))

df_clean.write.partitionBy("estado").mode("overwrite").parquet("s3://shopbr-datalake-joyce/silver/pedidos_limpos/")

print(f"Registros processados: {df_clean.count()}")
print("Job concluido com sucesso!")
```

AWS Glue Job Silver: Execution

Processing

job

Last modified on 12/04/2026, 12:23:29 | Actions | Save | Run

Script | Job details | Runs | Data quality | Schedules | Version Control

Job runs (1/10) Info

Last updated (UTC) April 14, 2026 at 14:52:22 | View details | Stop job run | Troubleshoot with AI | Table View | Card View

Filter job runs by property

1

Run status	Retries	Start time (Local)	End time (Local)	Duration	Capacity (DPUs)	Worker type	Glue version
Succeeded	0	04/14/2026 12:33:59	04/14/2026 12:35:27	1 m 18 s	2 DPUs	G.1X	5.0

Amazon S3: silver/pedidos_limpos/ particionado

Processing

shopbr-datalake-joyce > silver/ > pedidos_limpos/

pedidos_limpos/

Copy S3 URI

Objects | Properties

Objects (9)

Copy S3 URI | Copy URL | Download | Open | Delete | Actions | Create folder | Upload

Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more

Find objects by prefix

Show versions

1

Name	Type	Last modified	Size	Storage class
estado=AM/	Folder	-	-	-
estado=BA/	Folder	-	-	-
estado=GO/	Folder	-	-	-
estado=MG/	Folder	-	-	-
estado=PR/	Folder	-	-	-
estado=RJ/	Folder	-	-	-
estado=RS/	Folder	-	-	-
estado=SC/	Folder	-	-	-
estado=SP/	Folder	-	-	-

Gold Layer: Analytics

Analytics

What happens in this layer?

The Gold Glue Job reads the clean data from Silver and produces aggregations ready for the business team to consume via Athena. No cleaning transformations are done here, only aggregation. Gold answers questions like: which state generated the most revenue? How many orders were placed per month?

What does Gold produce?

- Reads data from Silver (pedidos_limpos/)
- Creates the month column from order_date in yyyy-MM format
- Groups by state and month
- Calculates total_sales (sum of value) and order_quantity (count)
- Saves to Parquet in gold/vendas_mensais/ with .mode('overwrite'): ensures there is no duplication

AWS Glue Job Gold: Script PySpark

Analytics

jobGold

Script | Job details | Runs | Data quality | Schedules | Version Control

Script Info

```python
import sys
from awsglue.context import GlueContext
from pyspark.context import SparkContext
from pyspark.sql.functions import col, sum, count, date_format

sc = SparkContext()
glueContext = GlueContext(sc)
spark = glueContext.spark_session

df_silver = spark.read.parquet("s3://shopbr-datalake-joyce/silver/pedidos_limpos/")

df_gold = df_silver \
    .withColumn("mes", date_format(col("data_pedido"), "yyyy-MM")) \
    .groupBy("estado", "mes") \
    .agg(
        sum("valor").alias("total_vendas"),
        count("pedido_id").alias("qtd_pedidos")
    ) \
    .orderBy("estado", "mes")

total = df_gold.count()
df_gold.write.mode("overwrite").parquet("s3://shopbr-datalake-joyce/gold/vendas_mensais/")
print(f"Registros gerados: {total}")
print("Job Gold concluido com sucesso!")
```

AWS Glue Job Gold: Execution

Analytics

jobGold

Last modified on 12/04/2026, 12:35:59 | Actions | Save | Run

Script | Job details | Runs | Data quality | Schedules | Version Control

Job runs (1/6) Info

Last updated (UTC) April 14, 2026 at 14:50:28 | View details | Stop job run | Troubleshoot with AI | Table View | Card View

Filter job runs by property

Run status	Retries	Start time (Local)	End time (Local)	Duration	Capacity (DPUs)	Worker type	Glue version
Succeeded	0	04/14/2026 12:38:40	04/14/2026 12:40:05	1 m 18 s	2 DPUs	G.1X	5.0

Amazon S3: gold/vendas_mensais/

Analytics

shopbr-datalake-joyce > gold/ > vendas_mensais/

vendas_mensais/

Copy S3 URI

Objects | Properties

Objects (1)

Copy S3 URI | Copy URL | Download | Open | Delete | Actions | Create folder | Upload

Objects are the fundamental entities stored in Amazon S3. You can use Amazon S3 inventory to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. Learn more

Find objects by prefix

Show versions

1

Name	Type	Last modified	Size	Storage class
part-00000-e997a68e-735a-4d4c-a99e-fd83f7cb453b-c000.snappy.parquet	parquet	April 14, 2026, 12:39:54 (UTC+01:00)	1.4 KB	Standard

Glue Crawler e Data Catalog

Cataloging

Two pieces with distinct purposes.

An important distinction: the Glue Job reads directly from S3; it doesn't depend on the Crawler. The Crawler exists for Athena. They are two independent automations.

Glue Crawler

- Scans S3 folders after each job
- Automatically detects file schema
- Registers tables in the Glue Data Catalog
- Without it, Athena cannot find the data
- 3 crawlers configured: bronze, silver, and gold

Glue Data Catalog

- Central index of all AWS account data
- Automatically created by AWS — unique per account
- Contains the created databases (shopbr_catalog_bronze, silver, gold)
- Tables are created by the Crawler when it runs
- Athena uses the Catalog to find and read the data

AWS Glue: Crawlers

Cataloging

Crawlers

A crawler connects to a data store, progresses through a prioritized list of classifiers to determine the schema for your data, and then creates metadata tables in your data catalog.

Crawlers (3) Info

Last updated (UTC)
April 14, 2026 at 14:53:22

Action | Run | Create crawler

View and manage all available crawlers.

Filter crawlers

1

Name	State	Schedule	Last run	Last run timestamp	Log	Table changes from last run
crawler-shopbr-bronze	Ready		Succeeded	April 12, 2026 at 12:00:25	View log	-
crawler-shopbr-gold	Ready		Succeeded	April 14, 2026 at 11:40:10	View log	-
crawler-shopbr-silver	Ready		Succeeded	April 14, 2026 at 11:35:30	View log	1 updated

Glue Data Catalog: Databases

Cataloging

Databases (5)

A database is a set of associated table definitions, organized into a logical group.

Last updated (UTC)
April 14, 2026 at 14:53:40

Edit | Delete | Add database

Choose catalog

276460385037

Filter databases

1

Name	Description	Location URI	Source catalog	Created on (UTC)
default	-	-	276460385037	April 14, 2026 at 09:44:08
shopbr_catalog	-	-	276460385037	April 2, 2026 at 22:47:46
shopbr_catalog_bronze	-	-	276460385037	April 12, 2026 at 11:43:18
shopbr_catalog_gold	-	-	276460385037	April 14, 2026 at 10:54:57
shopbr_catalog_silver	-	-	276460385037	April 14, 2026 at 09:59:14

Glue Data Catalog: Schema Bronze

Cataloging

table_pedidos

Last updated (UTC)
April 14, 2026 at 14:53:51

Version 0 (Current version) | Actions

Table overview | Data quality - *new*

Table details

Name
table_pedidos

Database
shopbr_catalog_bronze

Description
-

Last updated
April 14, 2026 at 09:47:24

Classification
-

Location
s3://shopbr-datalake-joyce/bronze/pedidos

Connection
-

Deprecated
-

Column statistics
No statistics

Advanced properties

Schema | Partitions | Indexes | Column statistics - *new*

Schema (8)

View and manage the table schema.

Edit schema as JSON | Edit schema

Filter schemas

1

#	Column name	Data type	Partition key	Comment
1	pedido_id	string	-	-
2	cliente_id	string	-	-
3	produto	string	-	-
4	categoria	string	-	-
5	valor	double	-	-
6	status	string	-	-
7	estado	string	-	-
8	data_pedido	string	-	-

Glue Data Catalog: Schema Silver

Cataloging

pedidos_limpos

Last updated (UTC)
April 14, 2026 at 14:54:39

Version 0 (Current version) | Actions

Table overview | Data quality - *new*

Table details

Name
pedidos_limpos

Database
shopbr_catalog_silver

Description
-

Last updated
April 14, 2026 at 10:29:02

Classification
-

Location
s3://shopbr-datalake-joyce/silver/pedidos_limpos

Connection
-

Deprecated
-

Column statistics
No statistics

▸ Advanced properties

Schema | Partitions | Indexes | Column statistics - *new*

Schema (8)

View and manage the table schema.

Edit schema as JSON | Edit schema

Filter schemas

1

#	Column name	Data type	Partition key	Comment
1	pedido_id	string	-	-
2	cliente_id	string	-	-
3	produto	string	-	-
4	categoria	string	-	-
5	valor	double	-	-
6	status	string	-	-
7	data_pedido	date	-	-
8	estado	string	Partition (0)	-

Glue Data Catalog: Schema Gold

Catalogação

vendas_mensais

Last updated (UTC)
April 14, 2026 at 14:54:17

Version 0 (Current version) | Actions

Table overview | Data quality - *new*

Table details

Name
vendas_mensais

Database
shopbr_catalog_gold

Description
-

Last updated
April 14, 2026 at 10:55:18

Classification
-

Location
s3://shopbr-datalake-joyce/gold/vendas_mensais

Connection
-

Deprecated
-

Column statistics
No statistics

▸ Advanced properties

Schema | Partitions | Indexes | Column statistics - *new*

Schema (4)

View and manage the table schema.

Edit schema as JSON | Edit schema

Filter schemas

1

#	Column name	Data type	Partition key	Comment
1	estado	string	-	-
2	mes	string	-	-
3	total_vendas	double	-	-
4	qtd_pedidos	bigint	-	-

Athena: SQL Queries

Consumption

How does Athena work in this pipeline?

Athena executes SQL queries directly on files stored in S3, without needing to load the data into a database. It uses the Glue Data Catalog to know where the data is located and what the schema of each table is.

Questions answered with SQL

Query 1: Sales by Category

Which category generates the most revenue? What is the average order value per category?

Query 2: Sales by State

Which Brazilian state concentrates the most sales and revenue?

Query 3: Orders by Status

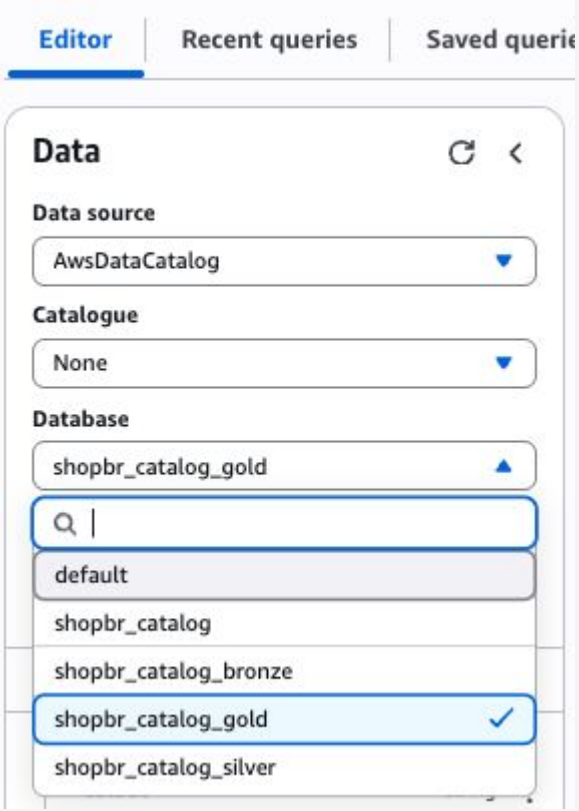
How many orders have been delivered, are in transit, or have been canceled?

Query 4: Top 5 Products

Which products sold the most and generated the most revenue?

Athena: Data

Consumption



Athena: Query 1 Sales by Category

Consumption

```sql
SELECT
    categoria,
    COUNT(*) AS total_pedidos,
    ROUND(SUM(valor), 2) AS receita_total,
    ROUND(AVG(valor), 2) AS ticket_medio
FROM shopbr_catalog_silver.pedidos_limpos
GROUP BY categoria
ORDER BY receita_total DESC;
```

SQL Ln 8, Col 29

Run again | Explain | Cancel | Clear | Create

Reuse query results
up to 60 minutes ago

Query results | Query status

Completed — Time in queue: 108 ms — Run time: 820 ms — Data scanned: 0.51 KB

Results (3)

Copy | Download results CSV

Search rows

1

#	categoria	total_pedidos	receita_total	ticket_medio
1	Eletrônicos	4	5996.9	1499.23
2	Moda	2	478.9	239.45
3	Utilidades	1	219.0	219.0

Athena: Query 2 Sales by State

Consumption

```sql
SELECT
    estado,
    COUNT(*) AS total_pedidos,
    ROUND(SUM(valor), 2) AS receita_total
FROM shopbr_catalog_silver.pedidos_limpos
GROUP BY estado
ORDER BY receita_total DESC
LIMIT 10;
```

SQL Ln 17, Col 10

Run again | Explain | Cancel | Clear | Create

Reuse query results
up to 60 minutes ago

Query results | Query status

Completed — Time in queue: 117 ms — Run time: 657 ms — Data scanned: 0.24 KB

Results (6)

Copy | Download results CSV

Search rows

1

#	estado	total_pedidos	receita_total
1	RS	1	3499.0
2	SP	2	1598.9
3	PR	1	899.0
4	RJ	1	349.0
5	MG	1	219.0
6	BA	1	129.9

Athena: Query 3 Orders by Status

Consumption

```sql
#pedidos por status
SELECT
    status,
    COUNT(*) AS total,
    ROUND(SUM(valor), 2) AS valor_total
FROM shopbr_catalog_bronze.table_pedidos
GROUP BY status;
```

SQL Ln 25, Col 40

Run | Explain | Cancel | Clear | Create

Reuse query results
up to 60 minutes ago

Query results | Query status

Completed — Time in queue: 121 ms — Run time: 441 ms — Data scanned: 0.58 KB

Results (3)

Copy | Download results CSV

Search rows

1

#	status	total	valor_total
1	entregue	6	3195.8
2	cancelado	1	189.9
3	em_transito	1	3499.0

Athena: Query 4 Top 5 Products

Consumption

```sql
#top 5 produtos

SELECT
    produto,
    categoria,
    COUNT(*) AS vezes_vendido,
    ROUND(SUM(valor), 2) AS receita
FROM shopbr_catalog_silver.pedidos_limpos
GROUP BY produto, categoria
ORDER BY receita DESC
LIMIT 5;
```

SQL Ln 35, Col 36

Run again | Explain | Cancel | Clear | Create

Reuse query results
up to 60 minutes ago

Query results | Query status

Completed — Time in queue: 103 ms — Run time: 752 ms — Data scanned: 0.81 KB

Results (5)

Copy | Download results CSV

Search rows

1

#	produto	categoria	vezes_vendido	receita
1	Notebook Pro	Eletrônicos	1	3499.0
2	Smartphone X	Eletrônicos	1	1299.9
3	Tablet 10	Eletrônicos	1	899.0
4	Tênis Running	Moda	1	349.0
5	Fone Bluetooth	Eletrônicos	1	299.0

Technical decisions

Engineering

Why was each choice made?

Manual polling on Lambda

Glue's native waiter blocks execution. Lambda has a 15-minute timeout, and Glue jobs can take longer. Polling with a configurable interval is more resilient.

Parquet instead of CSV

Athena charges per scanned data point. Parquet compresses and is columnar, reading only the necessary columns. Cost reduction of up to 90% compared to CSV.

State partitioning

A query filtering SP reads only state=SP, not the entire dataset. This is a performance and cost decision that scales well with increasing volume.

.mode('overwrite')

Ensuring idempotency means the pipeline can be executed without generating duplicates. This provides predictable and safe behavior for reprocessing.

Next steps

Evolution

What could be improved in this architecture?

Region sa-east-1

Migrate to São Paulo, lower latency in Brazil.

VPC e IAM granular

Private network and roles with minimum permissions per service.

Scheduled crawlers

Schedule CRON in Glue for automatic cataloging.

Jobs by domain

Separate pipelines for customers, products, and inventory.

Views no Athena

Saved queries ready for the business team.

CloudWatch Alerts

Automatic notifications in case of pipeline failure.

Thanks

Pipeline de Dados na AWS · ShopBR

Medallion Architecture · S3 · Lambda · Glue · Athena · CloudWatch

Joyce Mara · 2026